SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)
Assessment Tool Weightage: 35%

Dr. Hemavathi R

QUESTIONS		Total Marks: 50	
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	**5**
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	**5**
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	**5**
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	**5**
5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	**5**
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	**5**
7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	**5**
8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	**5**

9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	**5**
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	**5**

Code of Conduct:

I Sree B (192524023) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: Sree B

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

1. **Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.**

Pseudocode:

```
ALGORITHM QueryProcessing(SQL_Query)

BEGIN
// Step 1: Receive SQL query
INPUT SQL_Query

// Step 2: Parsing
Tokens ← LexicalAnalysis(SQL_Query)
ParseTree ← SyntaxAnalysis(Tokens)

IF ParseTree is INVALID THEN
DISPLAY "Syntax Error"
RETURN
END IF

// Step 3: Semantic Analysis
ValidatedTree ← SemanticAnalysis(ParseTree)

IF ValidatedTree is INVALID THEN
DISPLAY "Semantic Error"
RETURN
END IF

// Step 4: Query Transformation
LogicalPlan ← TransformToRelationalAlgebra(ValidatedTree)
OptimizedPlan ← OptimizeQuery(LogicalPlan)

// Step 5: Physical Plan Selection
ExecutionPlan ← SelectPhysicalOperators(OptimizedPlan)

// Step 6: Query Execution
Result ← Execute(ExecutionPlan)

// Step 7: Return result
DISPLAY Result
END
```

Flow:

SQL Input → Parsing → Semantic Checking → Query Transformation → Optimization → Physical Plan → Execution → Result Output

2. Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.

Pseudocode:

```
ALGORITHM CostBasedOptimization(Tables)

BEGIN
BestPlan ← NULL
MinimumCost ← INFINITY

JoinOrders ← GenerateAllJoinOrders(Tables)

FOR each Order IN JoinOrders DO
Cost ← 0
CurrentRelation ← FirstTable(Order)

FOR each NextTable IN RemainingTables(Order) DO
// Estimate number of pages
Pages1 ← EstimatePages(CurrentRelation)
Pages2 ← EstimatePages(NextTable)

// Estimate I/O cost of join
JoinCost ← Pages1 + (Pages1 × Pages2)

Cost ← Cost + JoinCost

CurrentRelation ← EstimateJoinResult(
CurrentRelation,
NextTable)
END FOR

IF Cost < MinimumCost THEN
MinimumCost ← Cost
BestPlan ← Order
END IF
END FOR

RETURN BestPlan, MinimumCost
END
```

Result:

The optimizer selects the join ordering having the **minimum estimated I/O cost**.

3. Write a pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.

Pseudocode:

```
ALGORITHM NestedLoopJoin(R, S)

BEGIN
Result ← EMPTY

FOR each tuple r IN R DO
  FOR each tuple s IN S DO
IF JoinCondition(r, s) = TRUE THEN
  Add (r, s) TO Result
  END IF
  END FOR
  END FOR

RETURN Result
END
```

Block Nested Loop Join using buffer pages:

```
ALGORITHM BlockNestedLoopJoin(R, S, B)

BEGIN
Result ← EMPTY

FOR each block of (B - 2) pages FROM R DO
  Load block into memory

  FOR each block of S DO
   Load S block into memory

  FOR each tuple r IN R block DO
    FOR each tuple s IN S block DO
      IF JoinCondition(r, s) THEN
        Add (r, s) TO Result
        END IF
      END FOR
    END FOR
  END FOR
END FOR

RETURN Result
END
```

Time/I/O Complexity:

If bR and bS are the number of pages in relations R and S, and B is the number of available buffer pages:

$$\text{Cost}=b_R+b_R/\lceil B-2b \rceil b_s$$

For tuple-level nested loop join, approximately:

$O(|R|\times|S|)$

where |R| and |S| are the numbers of tuples.

4. Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.

Pseudocode:

```
ALGORITHM DetectDeadlock(WFG)

BEGIN
FOR each transaction T IN WFG DO
    Mark T as UNVISITED
END FOR

FOR each transaction T IN WFG DO
    IF T is UNVISITED THEN
        IF DFS(T) = TRUE THEN
            DISPLAY "Deadlock Detected"
            RETURN TRUE
        END IF
    END IF
END FOR

DISPLAY "No Deadlock"
RETURN FALSE
END


FUNCTION DFS(T)

BEGIN
Mark T as VISITING

FOR each transaction U adjacent to T DO

    IF U is VISITING THEN
        RETURN TRUE       // Cycle found
    END IF

    IF U is UNVISITED THEN
        IF DFS(U) = TRUE THEN
            RETURN TRUE
        END IF
    END IF

END FOR

Mark T as VISITED
RETURN FALSE
END
```

Key point:

A cycle in the **Wait-For Graph** indicates that transactions are waiting for one another, so a **deadlock exists**.

5. Design pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.

Pseudocode:

```
ALGORITHM TwoPhaseLocking(Transaction T)

BEGIN
Phase ← GROWING

FOR each operation OP of T DO

IF OP is READ(X) OR OP is WRITE(X) THEN

IF LockAvailable(X, T) THEN
GrantLock(T, X)
ELSE
WAIT until LockAvailable(X, T)
GrantLock(T, X)
END IF

Execute(OP)
END IF

IF OP is UNLOCK(X) THEN

IF Phase = GROWING THEN
Phase ← SHRINKING
END IF

ReleaseLock(T, X)
END IF

END FOR

ReleaseAllLocks(T)
END
```

Two phases:

1. **Growing Phase:** Transaction can acquire locks but cannot release them.
2. **Shrinking Phase:** Transaction can release locks but cannot acquire new locks.

This ensures **conflict serializability**.

6. Write a pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.

Pseudocode:

```
ALGORITHM ThomasWriteRule(T, Operation, X)

BEGIN
TS ← Timestamp(T)

IF Operation = READ(X) THEN

IF TS < WriteTimestamp(X) THEN
ABORT T
RETURN
ELSE
Perform READ(X)
ReadTimestamp(X) ← MAX(ReadTimestamp(X), TS)
END IF

ELSE IF Operation = WRITE(X) THEN

IF TS < ReadTimestamp(X) THEN
ABORT T
RETURN

ELSE IF TS < WriteTimestamp(X) THEN
// Ignore obsolete write
IGNORE WRITE(X)
RETURN

ELSE
Perform WRITE(X)
WriteTimestamp(X) ← TS
END IF

END IF
END
```

Thomas' Write Rule:

If a write operation is obsolete because a newer write has already occurred, the old write can be **ignored instead of aborting the transaction**.

7. Write a pseudocode algorithm for the ARIES recovery technique with Analysis, Redo, and Undo phases.

```
ALGORITHM ARIES_Recovery(Log)

BEGIN

// Phase 1: Analysis
TransactionTable ← EMPTY
DirtyPageTable ← EMPTY

FOR each LogRecord IN Log DO

IF LogRecord is TRANSACTION_START THEN
    Add transaction to TransactionTable
END IF

IF LogRecord modifies a page THEN
    Add page to DirtyPageTable
END IF

IF LogRecord is COMMIT or ABORT THEN
    Update TransactionTable
END IF

END FOR

// Phase 2: Redo
StartLSN ← MinimumRecoveryLSN(DirtyPageTable)

FOR each LogRecord from StartLSN DO
    IF RedoRequired(LogRecord) THEN
        REDO LogRecord
    END IF
END FOR


// Phase 3: Undo
FOR each incomplete transaction T DO
    WHILE T has undoable log records DO
        LogRecord ← GetLastLogRecord(T)
        UNDO LogRecord
        WriteCompensationLogRecord()
    END WHILE

    WriteAbortRecord(T)
END FOR

DISPLAY "Recovery Completed"
END
```

ARIES phases:

- **Analysis:** Determines active transactions and dirty pages.
- **Redo:** Repeats necessary operations to restore database state.
- **Undo:** Rolls back incomplete transactions.

8. Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.

Pseudocode:

```
ALGORITHM ShadowPaging

BEGIN
ShadowPageTable ← Copy(CurrentPageTable)

WHILE Transaction is ACTIVE DO

Page ← GetPageToModify()

NewPage ← AllocateNewPage()

Copy Page contents INTO NewPage

Modify(NewPage)

CurrentPageTable[Page] ← NewPage

END WHILE

IF Transaction COMMIT succeeds THEN
Save CurrentPageTable
Delete Old Pages
ELSE
CurrentPageTable ← ShadowPageTable
Free Modified Pages
DISPLAY "Transaction Rolled Back"
END IF
END
```

Key idea:

The **shadow page table remains unchanged** during transaction execution. If a failure occurs, the system restores the shadow page table.

9. Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.

Pseudocode:

```
ALGORITHM ImmediateUpdateRecovery(Log)

BEGIN
ActiveTransactions ← FindActiveTransactions(Log)
CommittedTransactions ← FindCommittedTransactions(Log)

// REDO phase
FOR each LogRecord IN Log DO
IF Transaction(LogRecord) IN CommittedTransactions THEN
REDO LogRecord
END IF
END FOR

// UNDO phase
FOR each LogRecord IN Log in reverse order DO
IF Transaction(LogRecord) IN ActiveTransactions THEN
UNDO LogRecord
END IF
END FOR

WriteRecoveryCompleteRecord()

DISPLAY "Database Recovery Completed"
END
```

Explanation:

- **REDO:** Reapplies changes of committed transactions.
- **UNDO:** Removes changes made by incomplete transactions.
- **The log** contains sufficient information to restore the database after a crash.

10. Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.

Pseudocode:

```
ALGORITHM DatabaseConnectivityManager

BEGIN

// Initialize connection pool
ConnectionPool ← CreateConnectionPool(MAX_CONNECTIONS)

FUNCTION ExecuteQuery(SQL, Parameters)

Connection ← NULL
Statement ← NULL

TRY
// Get connection from pool
Connection ← ConnectionPool.GetConnection()

Connection.SetAutoCommit(FALSE)

// Prepare and execute query
Statement ← Connection.PrepareStatement(SQL)
SetParameters(Statement, Parameters)

Result ← Statement.Execute()

// Commit successful transaction
Connection.Commit()

RETURN Result

CATCH DatabaseException E

// Roll back failed transaction
IF Connection ≠ NULL THEN
Connection.Rollback()
END IF

DISPLAY "Database Error: ", E.Message
RETURN FAILURE

FINALLY

IF Statement ≠ NULL THEN
Statement.Close()
END IF

IF Connection ≠ NULL THEN
ConnectionPool.ReturnConnection(Connection)
END IF
```

END TRY
END FUNCTION

END

Main components:

1. **Connection Pooling** – Reuses database connections.
2. **Query Execution** – Prepares and executes SQL statements.
3. **Commit** – Saves successful transactions.
4. **Rollback** – Reverses changes when an exception occurs.
5. **Connection Return** – Returns the connection to the pool for reuse.